

Fiche recap'

Check the code and ask questions

• Basic HTTP server :

Un serveur HTTP fonctionne selon le modèle **client-serveur**.

Le serveur crée un **socket** (`socket ( )`), le lie à un port (`bind ( )`), et écoute les connexions (`listen ( )`)

Un client (navigateur) se connecte → le serveur **accepte** la connexion (`accept ( )`) → un nouveau fd est créé

Le client envoie une **requête HTTP** (méthode, headers, body) → le serveur la parse

Le serveur envoie une **réponse HTTP** (status code, headers, body)

La connexion est fermée ou réutilisée (keep-alive)

• Fonction utilisée pour multiplexing :

`poll ( )` permet de surveiller plusieurs file descriptors simultanément sans bloquer le programme. C'est ce qui permet de gérer plusieurs clients en parallèle dans un seul thread.

• Comment poll() fonctionne:

Explication prototype de la fonction

On lui passe un tableau de **pollfd**, chaque entrée contenant

fd : le file descriptor à surveiller

events : les événements qu'on veut surveiller (POLLIN, POLLOUT)

revents : les événements qui se sont **réellement produits** (rempli par le kernel)

`poll ( )` **bloque** jusqu'à ce qu'au moins un fd soit prêt (ou que le timeout expire). On itère ensuite sur le tableau et on traite chaque fd dont **revents** est non nul.

Difference avec select :

Pas de limite à 1024 connexions car prend un tab dynamique

• Only poll() pour accept et read/write

Oui un seul poll() dans la boucle principale qui gère tout.

Les **fds serveurs** (sockets d'écoute) sont dans le tableau → quand POLLIN est détecté sur eux, on appelle `acceptClient ( )` pour accepter une nouvelle connexion

Les **fds clients** sont ajoutés dynamiquement → POLLIN = lire la requête, POLLOUT = envoyer la réponse

Tout passe par le **même appel poll ()** dans `pollLoop ( )`

• poll() dans la main loop, vérifie READ et WRITE en même temps

Une seule iteration . `poll ( )` est appelé une seule fois par tour de boucle et surveille simultanément POLLIN et POLLOUT sur tous les fds.

• Un seul read/write par client par poll ()

Un seul POLLIN → un seul `handleRequest()` ou `acceptClient()`. Check ds la boucle poll()

Pour chaque fd, on ne lit qu'une fois et on n'écrit qu'une fois par itération de `poll ( )`

• Si erreur, client is removed

Oui, check **recv()** dans **handleRequest()**
aussi **send()** dans **sendResponse()**

- **Valeur de retour correct (pas juste 0 ou -1)**

Oui dans recv() les 2 cas sont verif séparément.
Si valeur positive on a traitement normal

- **Est-ce que **errno** est vérifié après **recv/send** ?**

Un errno au debut de la poll loop donc ok. Les autres ont été supp

- **Aucun read/write sans passer par **poll()****

Tous les **recv/send** sur les sockets clients sont appelés **uniquement depuis**
handleRequest() et **sendResponse()**, qui sont eux-mêmes appelés **uniquement**
depuis pollLoop(), après détection de POLLIN/POLLOUT par **poll()**

Configuration

- **HTTP response status codes list**

Fichier txt dispo dans le projet

- **multiples server with different port**

Oui, ds le fichier conf on a 8080, 8081, 8082, 8083, 8084, 9090

- **Plusieurs serveurs avec des hostnames différents (Virtual Host)**

Oui, on a 8080 avec **server_name** et **vhost.com**

```
printf "GET / HTTP/1.1\r\nHost: localhost\r\nConnection: close\r\n\r\n" | nc 127.0.0.1 8080
```

```
printf "GET / HTTP/1.1\r\nHost: [vhost.com](<http://vhost.com/>)\r\nConnection: close\r\n\r\n" | nc 127.0.0.1 8080
```

- **page d erreur personnalisé**

Oui, **error_page 404 /errors/404.html**; si on fait :

```
curl <http://127.0.0.1:8080/page-qui-nexiste-pas>  
va retourner notre 404 page perso
```

- **body limit**

Oui, plusieurs serveurs avec des body limit diff :

Serveur 1 : limite = 100 octets → doit rejeter (413)

```
head -c 10000000 /dev/zero | \ curl -X POST <http://localhost:8080/upload/big.txt> --data-binary @-
```

Serveur 4 (8081) : limite = 1 000 000 000 → doit accepter (200 ok)

```
head -c 10000000 /dev/zero | \ curl -X POST <http://
```

```
localhost:8081/upload/big.txt> --data-binary @-
```

• Routes vers des répertoires différents

Oui, on a bien :

/ → ./www

/upload → ./www/upload

/cgi-bin → ./www/cgi-bin (avec CGI activé)

/old-site → redirect 301 vers /

les commandes :

```
curl <http://127.0.0.1:8080/> curl <http://  
127.0.0.1:8080/upload> curl <http://127.0.0.1:8080/old-  
site>
```

• Fichier par défaut pour un répertoire

Oui, index index.html pour tous les serveurs, sauf le virtual host qui a un diff avec vhost.html

• Méthodes acceptées par route

Check avec le serveur 8083 car méthodes limitées

/upload n'accepte que POST sur 8083 (donc 405 methode not allowed)

```
curl -X GET <http://127.0.0.1:8083/upload> curl -X DELETE  
<http://127.0.0.1:8083/upload>
```

mais fonctionne pour

```
curl -v -X POST <http://localhost:8080/upload/a.txt> \ -d  
"hello"
```

Basic checks

Liste de tous les test dans doc séparé :

<https://app.notion.com/p/Testing-pour-eval-3775526d716b80acb4cbea732a57deda>

Check CGI

Liste de tous les test dans doc séparé :

<https://app.notion.com/p/Testing-pour-eval-3775526d716b80acb4cbea732a57deda>

Check with a browser

Page normale <http://127.0.0.1:8080/>

URL incorrecte (404) <http://127.0.0.1:8080/nope>

Listing répertoire <http://127.0.0.1:8080/upload/>

Redirection 301 <http://127.0.0.1:8080/old-site>

Port issues

Multiple ports et website different : Oui, chaque serveur a des spe differentes

Siege & stress test

Siege

```
siege -b <http://127.0.0.1:8084/>
```

No leaks

No hanging connexion

Commande Valgrind :

```
valgrind --leak-check=full --show-leak-kinds=all --track-  
fds=yes --log-file=valgrind.log ./Webserv config/  
config.conf
```